Advanced Topics in Databases 2025/26

Instructor

Michel Ferreira (Office 1.22)
Email: mpferrei@fc.up.pt

Communication channel

- Moodle: materials repository, group registration and assignment submission

Course format

- 1.5h lecture + 2h lab per week
- Lecture: Fri 09:30–11:00
- Lab: Fri 11:00–13:00 (starts next week)
- Strong hands-on component.
- Independent study and continuous work are required.

Syllabus

Lectures:

- Advanced SQL & analytics in PostgreSQL (windows, recursion, GROUPING SETS, CUBE/ROLLUP)
- Performance & indexing (EXPLAIN/ANALYZE; B-tree, GIN, GiST, BRIN; partial/functional indexes)
- Server-side programming: PL/pgSQL (functions, procedures, triggers; auditing & data integrity)
- Spatial databases with PostGIS (types, SRIDs, operators, spatial joins, spatial indexing)
- Python integration and spatial visualisation over map tiles
- Data warehousing & OLAP in PostgreSQL (star schema; materialized views; multidimensional operators)
- Vector databases in PostgreSQL with pgvector (embeddings, similarity search, indexing) and LLM workflows (RAG)

Syllabus

Labs:

- Weekly exercise sheets (few exercises, increasing complexity)
- Exercises are connected across weeks—keep up with the thread
- Some lab sessions are used for project supervision and feedback

Project

- Group project (2–3 students)
- Single final submission at the end of the semester
- Topic: municipal elections analytics (2021 & 2025) using PostgreSQL/PostGIS + Python

Assessment

- Project (group): 40%
- Groups of 2–3 students; group registration on Moodle
- Project presentation: 10%
- Exam: 50%
- Minimum grade in exam: 8/20

Bibliography

- Fontaine, Dimitri. The Art of PostgreSQL. Lulu.com, 2019. ISBN: 9781949950366
- Obe, Regina; Hsu, Leo. PostGIS in Action (3rd ed.). Manning, 2021. ISBN: 9781617296697
- Kimball, Ralph; Ross, Margy. The Data Warehouse Toolkit (3rd ed.). Wiley, 2013. ISBN: 9781118530801
- pgvector documentation (official project README: pgvector/pgvector on GitHub)
- PostgreSQL documentation; PostGIS documentation
- psycopg (Python driver) documentation; GeoPandas and mapping libraries documentation

PostgreSQL (Postgres)

- **Website:**
  - https://www.postgresql.org/
- **What it is (short description):**
  - PostgreSQL is a powerful, open-source relational database management system (RDBMS) known for standards compliance, extensibility, and reliability. It supports advanced SQL, robust transactions (ACID), rich indexing options, and a large ecosystem of extensions.
- **Brief history:**
  - Originates from the **POSTGRES** research project at the University of California, Berkeley (late 1980s).
  - Renamed **PostgreSQL** in the mid-1990s when SQL support was added.
  - Since then, it has been developed by a global open-source community and is widely adopted in both industry and academia.

Why PostgreSQL for this course:

- **Advanced SQL is first-class**: window functions, recursion, grouping sets/rollups/cubes, rich aggregates.
- **Extensible by design**: add capabilities via extensions rather than switching databases.
- **Performance engineering is teachable**: excellent tooling (EXPLAIN (ANALYZE, BUFFERS)), multiple index types (B-tree, GIN, GiST, BRIN), partitioning, materialized views.
- **One consistent platform across modules**: relational + spatial + analytical + vector search, reducing overhead and enabling deeper learning.

Significant real-world use cases:

- **Transactional systems** (web apps, fintech/ERP/CRM) needing strong consistency and reliability.
- **Analytics & reporting** (data marts, OLAP-style queries, materialized views).
- **Geospatial applications** (urban planning, logistics, mapping) via PostGIS.
- **Hybrid search** combining structured filters with full-text search and vector similarity (modern AI/RAG pipelines).

Significant real-world use cases:

- **Instagram** — published scaling notes specifically about running and growing with Postgres ("Handling Growth with Postgres").
- **Netflix** — describes migrating/standardising relational workloads on **Aurora PostgreSQL** (PostgreSQL-compatible) and automation around Postgres migrations.
- **OpenStreetMap** — core rendering/import tooling is built around PostgreSQL + PostGIS (e.g., osm2pgsql → PostgreSQL/PostGIS).

Instagram **Netflix** **OpenStreetMap**

Extensions / modules used in this course:

- **PostGIS** — spatial types, spatial functions, spatial indexing
- **pgRouting** — Adds **routing and network analysis** (shortest path, k-shortest paths, TSP, isochrones) on top of **PostGIS** for road-network queries in PostgreSQL.
- **pgvector** — embeddings, similarity search, vector indexes
- (Plus core PostgreSQL features: PL/pgSQL, advanced indexing, EXPLAIN, materialized views)

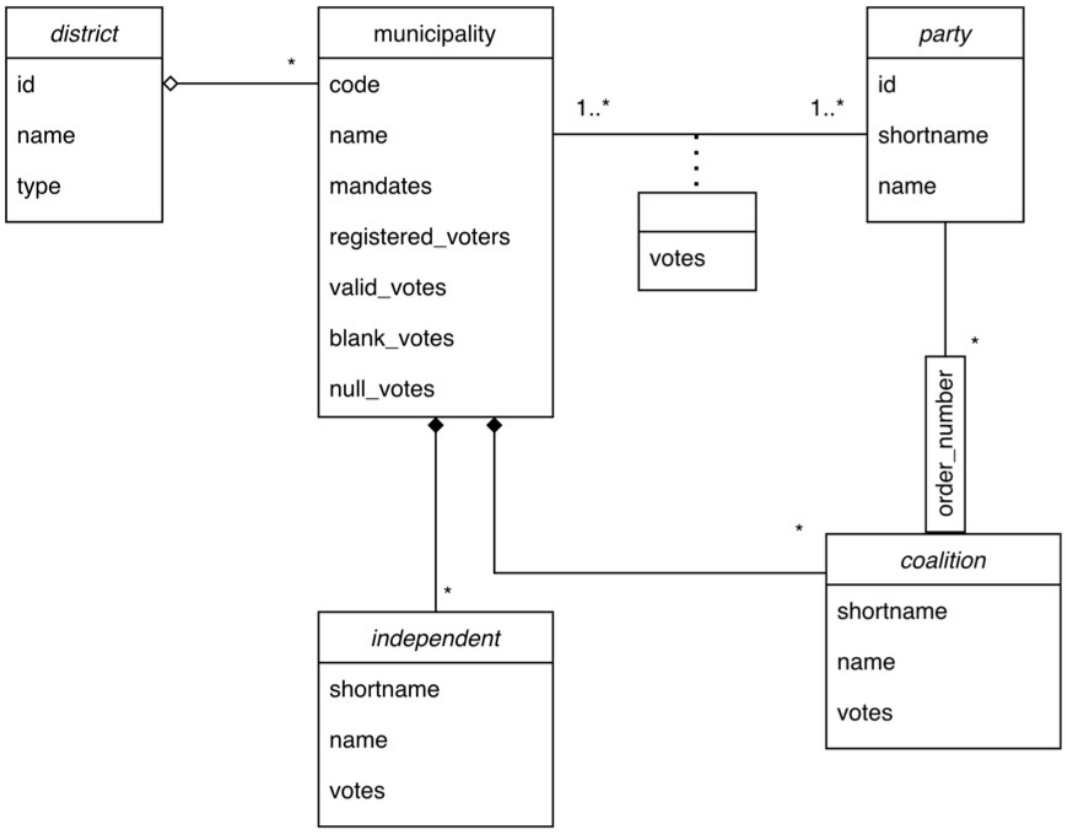
Next week's assignment: Install PostgreSQL

- Before next week's class, please install PostgreSQL on your laptop.
- Platform-independent: you can install it on Linux, macOS, or Windows.
- Make sure you can:
  - start the PostgreSQL service,
  - connect to it (e.g., with psql or a GUI such as pgAdmin),
  - create a database and run a simple query.
- We will use PostgreSQL throughout the course (and later add extensions such as PostGIS, pgvector, and pgRouting).

The Portuguese Local Elections 2021(City Council) Database

- This database is created in the context of the local elections (Autárquicas) for the City Council only, that were held in Portugal on 26th of September of 2021.
- In each municipality, lists can be:
  - Parties (national parties, stored in party);
  - Coalitions (sets of parties defined per municipality, stored in coalition + coalition_member); Same party set may have different names in other municipalities.
  - Citizens' Groups (GCE), modeled as independent lists per municipality (independent). Exist only in that municipality.

Conceptual Model



Relational Schema

```
district(id,name,type)
name UK
type ∈ {'District','Autonomous Region'}

municipality(code,name,district->district(id),mandates,registered_voters,val
id_votes,blank_votes,null_votes)

party(id,shortname,name)
shortname UK

independent(municipality_code->municipality(code),shortname,name,votes)

coalition(municipality_code->municipality(code),shortname,name,votes)

coalition_member(municipality_code->coalition.municipality_code,
coalition_shortname->coalition.shortname,order_number,party_id->party(id))

party_votes(municipality_code->municipality(code),party_id->party(id),votes)
```

Create the database and load the dataset

- We will use a shared PostgreSQL dataset in the lab classes.
- To prepare your environment, you must:
  - Create a new PostgreSQL database
  - Load the relational schema + data (tuples) from the SQL file available in Moodle.

Commands (terminal)

```
# 1) Create the database
createdb aut2021

# 2) Load schema + data from the SQL file (replace with the real filename)
psql -d aut2021 -f aut2021.sql
```

Example of more complex SQL queries

Write a **single SQL query** in PostgreSQL that computes the **distribution of mandates** for the municipality of **Porto** using the **D'Hondt method**.

Your query must:

1. Select the municipality record where `name = 'PORTO'` (case-insensitive) and obtain its number of mandates.
2. Build the set of competing lists in Porto by combining:
   - parties (`party_votes` joined with `party`),
   - coalitions (`coalition`),
   - independents (`independent`).
3. Generate the D'Hondt divisors from **1** to the number of mandates.
4. For each list, compute the sequence of quotients:

$$q_k = \frac{\text{votes}}{k} \quad \text{for } k = 1..(\text{mandates})$$

5. Rank all quotients across all lists in **descending order** and select the top **N** quotients, where **N** is the number of mandates of Porto.
6. Return the final result as a table with one row per list containing:
   - `list_type` (party / coalition / independent),
   - `shortname` ,
   - `list_name` ,
   - total `votes` ,
   - number of `mandates` allocated to that list.

Order the output by:

- number of mandates (descending),
- votes (descending),
- and then `shortname` .

```
  list_type   | shortname |                    list_name                    | votes | mandates
--------------+-----------+-------------------------------------------------+-------+----------
 independent  | RM        | "Rui Moreira: Aqui há Porto"                    | 41213 |        6
 party        | PS        | Partido Socialista                              | 18192 |        3
 party        | PPD/PSD   | Partido Social Democrata                        | 17481 |        2
 party        | PCP-PEV   | CDU - Coligação Democrática Unitária (PCP-PEV)  |  7610 |        1
 party        | BE        | Bloco de Esquerda                               |  6321 |        1
(5 rows)
```

How the D'Hondt method works

- For each candidacy, compute a series of quotients by dividing its vote total by the integers 1, 2, 3, …, S, where S is the number of mandates (seats).
- Pool all quotients together and pick the top S quotients overall.
- Each time a candidacy's quotient is picked, that candidacy receives one seat.

SQL WITH RECURSIVE and WINDOW Functions

```sql
WITH RECURSIVE
mandate AS (
  SELECT code, name, mandates::int AS mandates
  FROM municipality
  WHERE upper(name) = 'PORTO'
),

lists AS (
  SELECT md.code AS municipality_code, 'party'::text AS list_type,
         p.shortname, p.name AS list_name, pv.votes::bigint AS votes
  FROM mandate md
  JOIN party_votes pv ON pv.municipality_code = md.code
  JOIN party p        ON p.id = pv.party_id

  UNION ALL
  SELECT md.code, 'coalition'::text, c.shortname, c.name, c.votes::bigint
  FROM mandate md
  JOIN coalition c ON c.municipality_code = md.code

  UNION ALL
  SELECT md.code, 'independent'::text, i.shortname, i.name, i.votes::bigint
  FROM mandate md
  JOIN independent i ON i.municipality_code = md.code
),

div(n) AS (
  SELECT 1
  UNION ALL
  SELECT n + 1
  FROM div
  WHERE n < (SELECT mandates FROM mandate)
),
```

```sql
scores AS (
  SELECT
    l.list_type,
    l.shortname,
    l.list_name,
    l.votes,
    d.n AS divisor,
    (l.votes::numeric / d.n::numeric) AS quotient
  FROM lists l
  JOIN div d ON true
  WHERE l.votes > 0
),

ranked AS (
  SELECT
    s.*,
    row_number() OVER (
      ORDER BY
        s.quotient DESC,
        s.votes DESC,
        s.shortname ASC,
        s.list_type ASC
    ) AS seat_number
  FROM scores s
),

winners AS (
  SELECT r.*
  FROM ranked r
  JOIN mandate m ON true
  WHERE r.seat_number <= m.mandates
)

SELECT
  w.list_type,
  w.shortname,
  w.list_name,
  max(w.votes) AS votes,
  count(*)     AS mandates
FROM winners w
GROUP BY w.list_type, w.shortname, w.list_name
ORDER BY mandates DESC, votes DESC, shortname ASC;
```